AWS KMS & Secrets Management

Importance of KMS & Secrets Management

AWS KMS

- Protects sensitive data using strong encryption keys.
- Ensures compliance (ISO, HIPAA, PCI-DSS).
- Gives centralized control over who can encrypt/decrypt data.
- Prevents data exposure if storage is compromised.

AWS Secrets Manager

- Eliminates hardcoded passwords and API keys.
- Automatically rotates credentials without app downtime.
- Controls access using IAM (least privilege).
- Audits secret access via CloudTrail.

🛠️ **Tools Used in This Implementation**

- AWS KMS – Key creation, rotation, access control
- Amazon S3 – Encrypted object storage
- AWS Secrets Manager – Secure credential storage
- IAM – Role and permission management
- CloudTrail – Logging and auditing access
- EC2 / Lambda – Application access testing

---

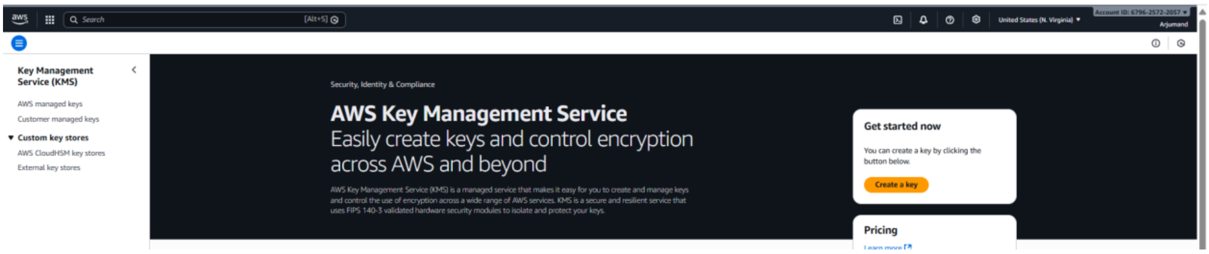
1. **How do you encrypt an S3 bucket using a customer-managed KMS key?**

Steps to Follow

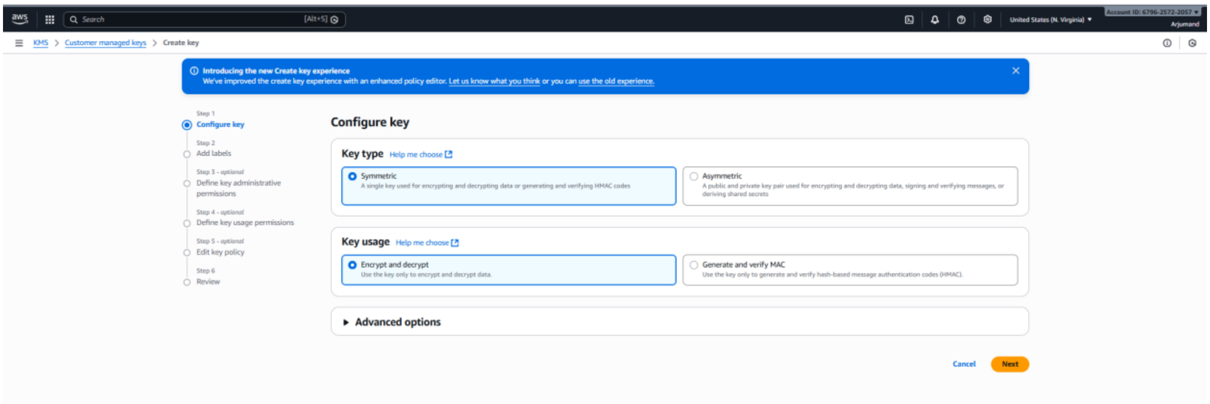
1. Create a symmetric Customer-Managed KMS key in AWS KMS.
2. Enable automatic key rotation for compliance.

3. Create or select an S3 bucket in the same region.
4. Enable default encryption on the bucket.
5. Choose SSE-KMS as encryption type.
6. Select the customer-managed KMS key (CMK).
7. Upload an object and verify it shows aws:kms encryption.

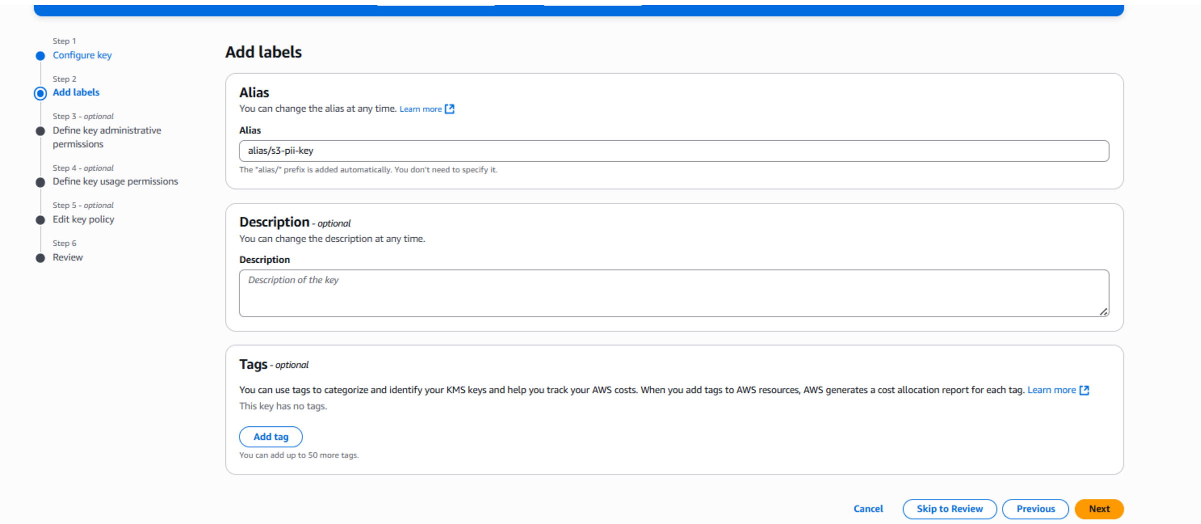
Go to AWS console and search for KMS and navigate to KMS



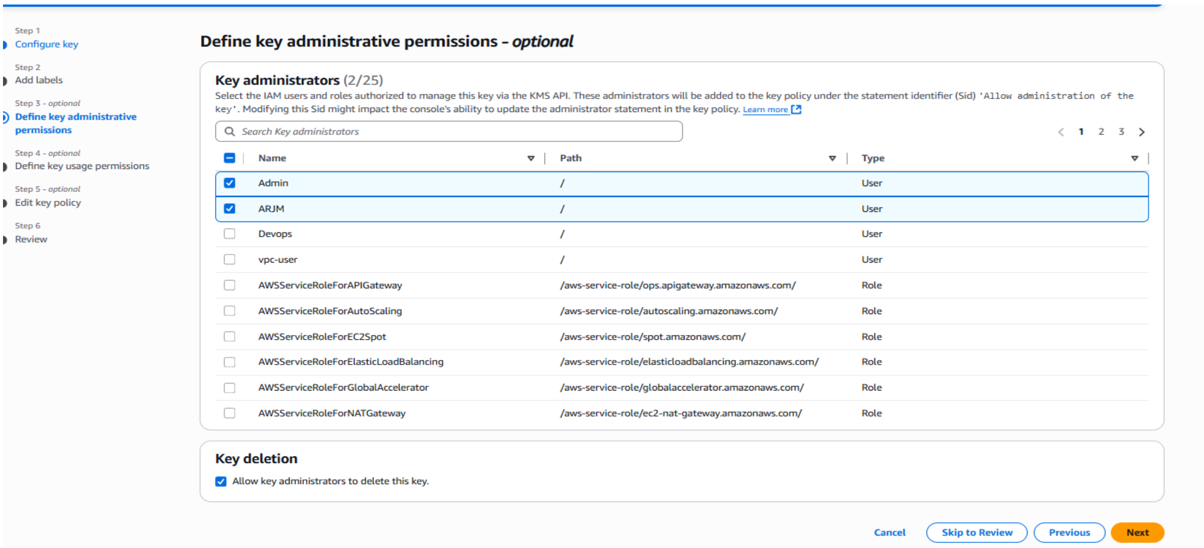
Select N.verginia region and click to create a key and choose option symmetric key, it will give same key for encryption and decryption.



Given key name

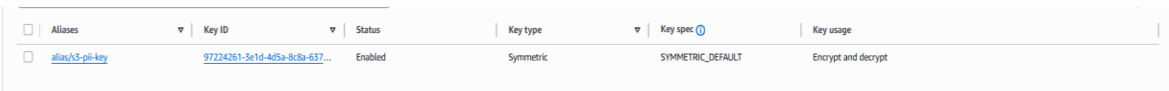


I have given permission to this IAM users

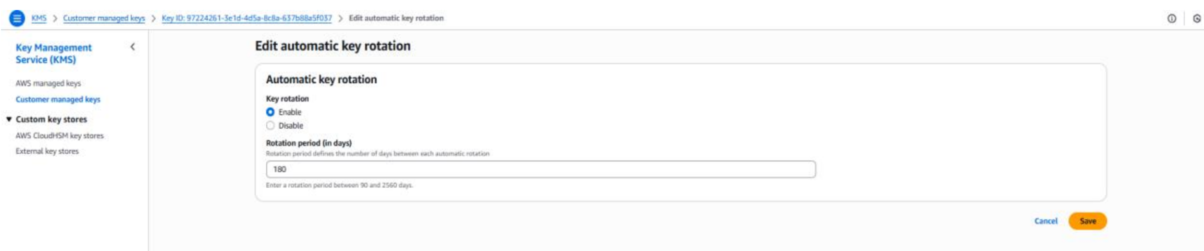


I have kept policy as it is as of now.

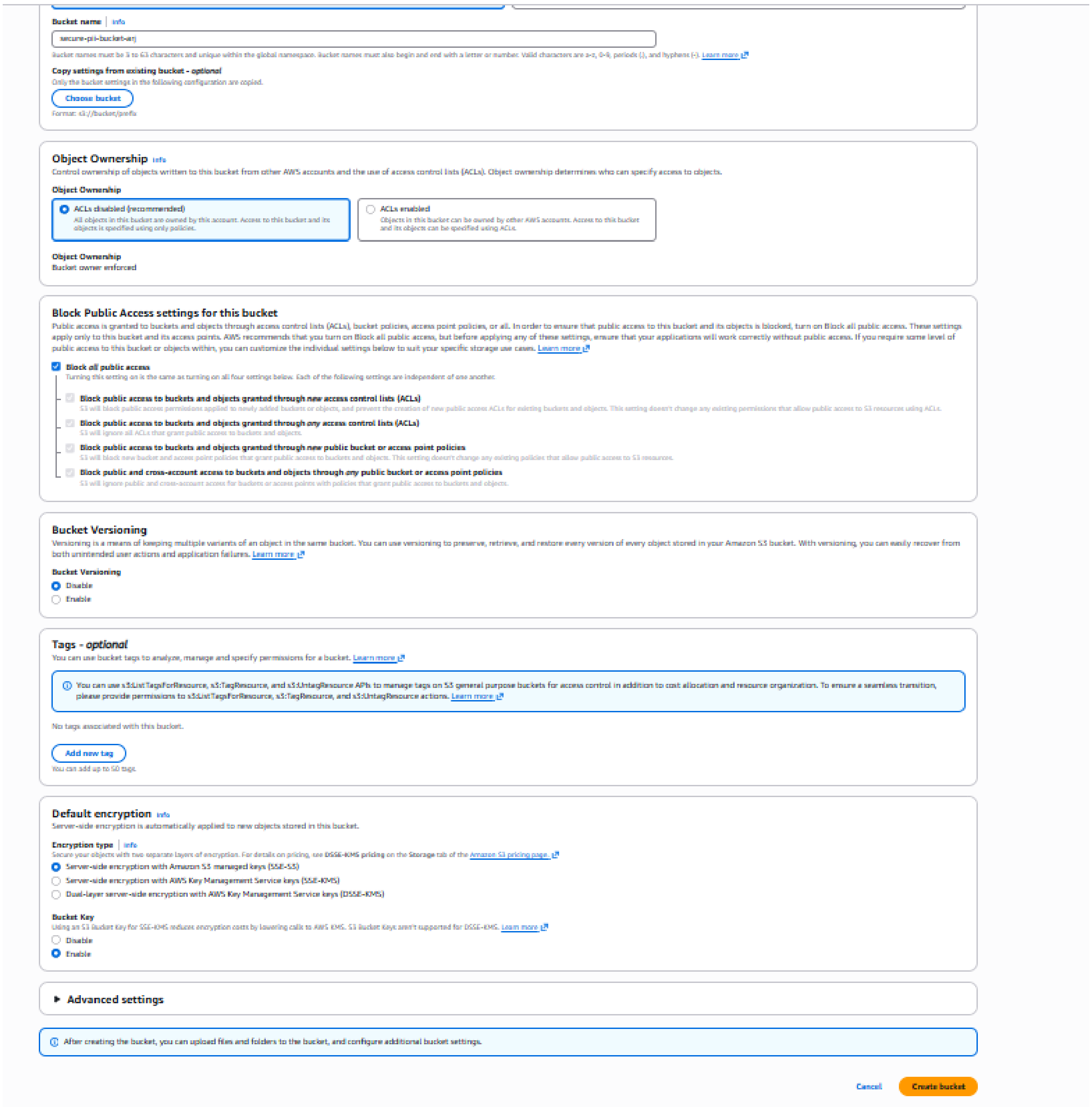
I have created one KMS (key management service)



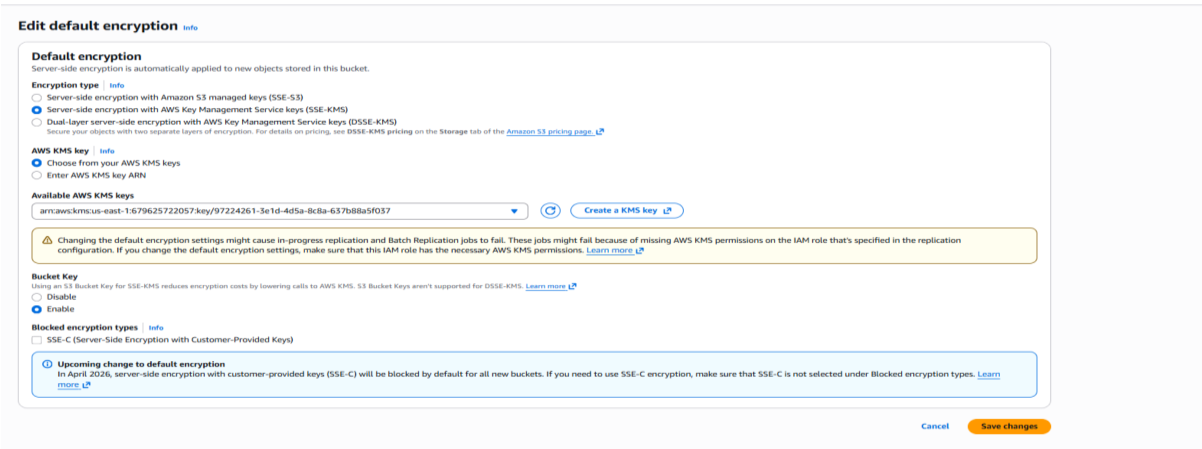
Enabled rotation and provided 180 days rotation time



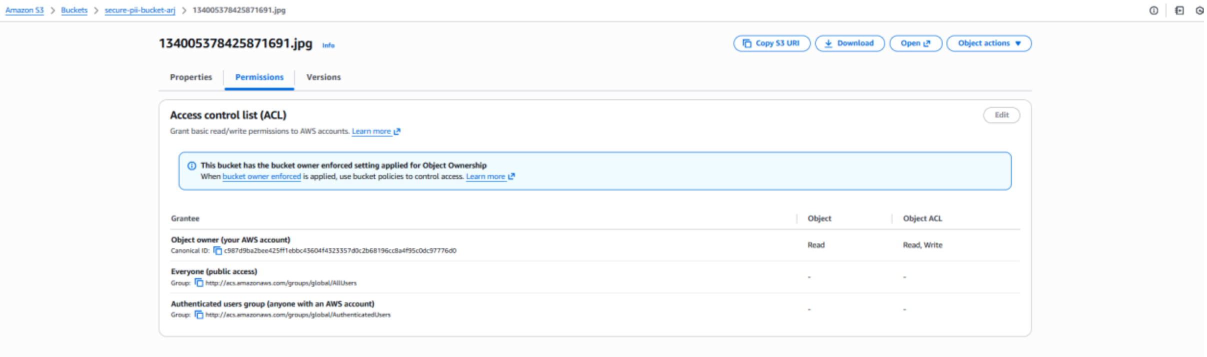
Created one bucket disable public access

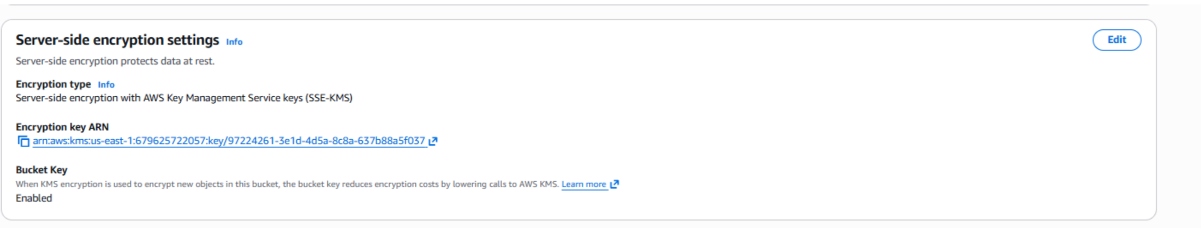


Enabled encryption and provided key name



We can confirm that there is encryption role is attached to the bucket





KMS TASK 2-How do you encrypt EBS volumes for EC2 using a customer-managed KMS key and verify it?

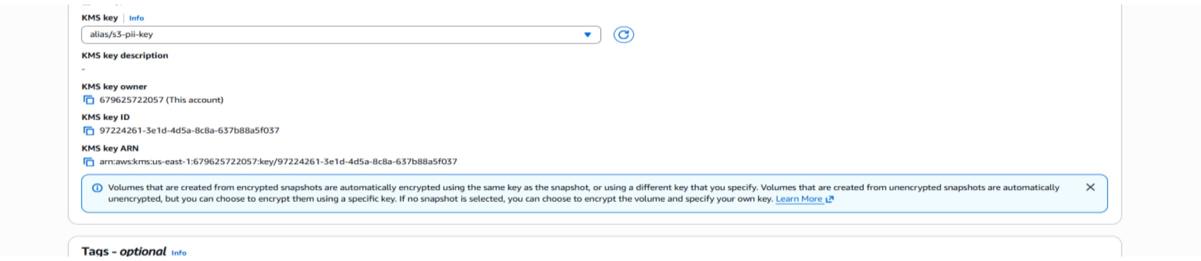
✅ Steps to Follow

1. Create or reuse a customer-managed KMS key for EBS encryption.
2. Enable automatic key rotation on the KMS key.
3. Go to EC2 → Volumes → Create volume.
4. Enable Encryption and select the CMK.
5. Attach the encrypted volume to an EC2 instance.
6. Verify the volume shows Encrypted: Yes with the CMK.
7. Create an encrypted snapshot from the volume.

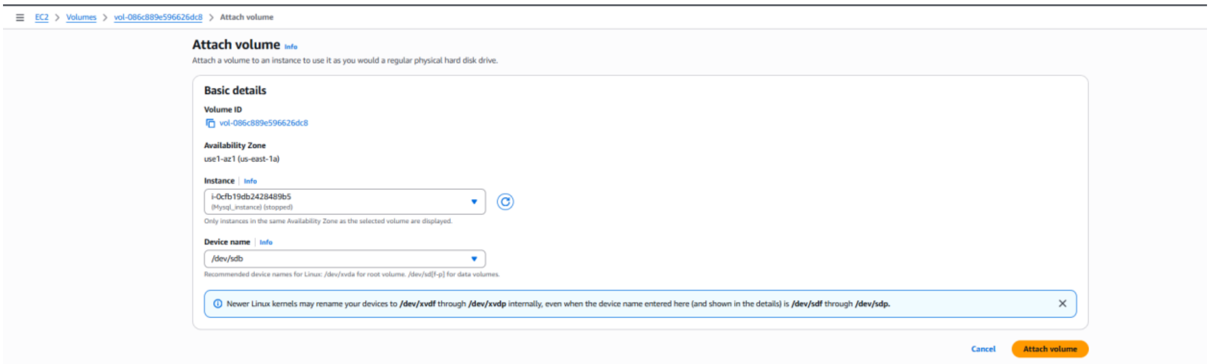
🎯 What this proves in real companies

- Disks are protected even if copied or stolen
- Meets security & compliance audits
- Enforces data-at-rest encryption standard

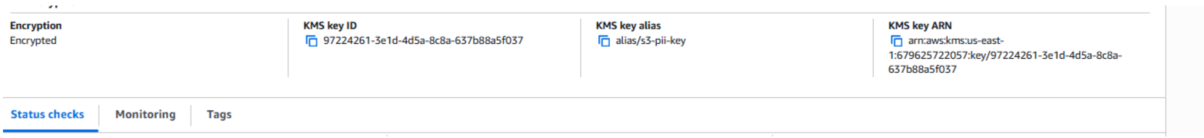
Created a volume and added encryption kms key



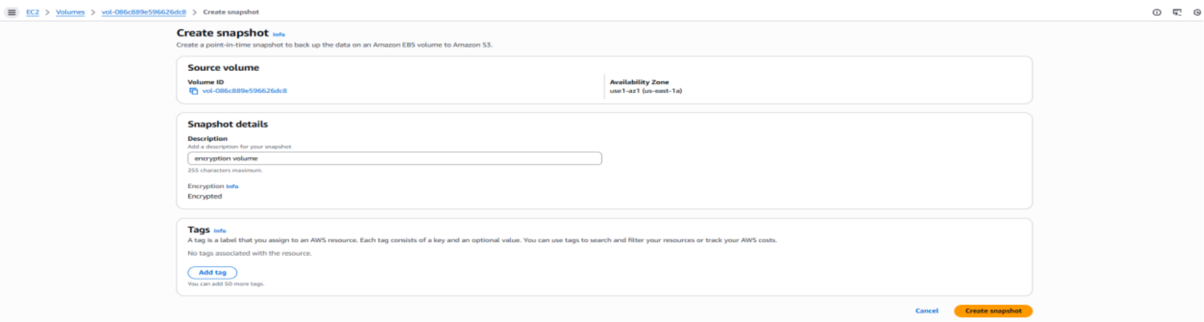
Attach that volume to the ec2



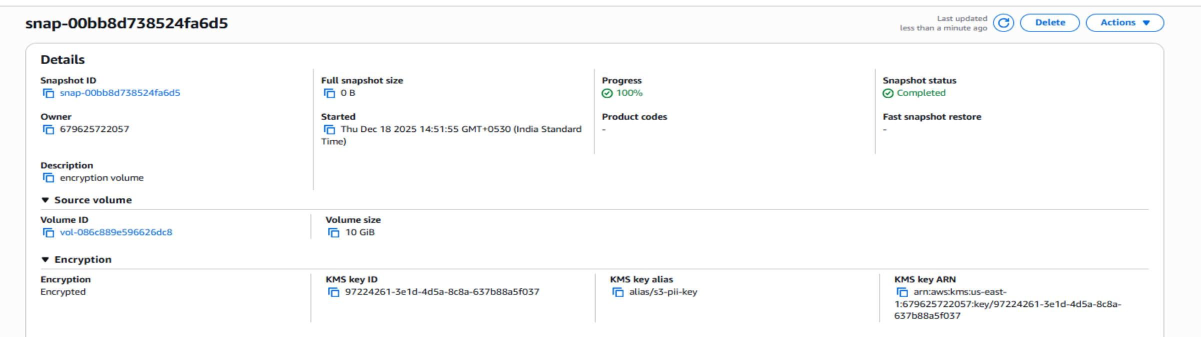
We can see volume is successfully attached to the KMS key



Created snapshot using the secured volume which I have added to kms key



Snapshot also attached to the kms key

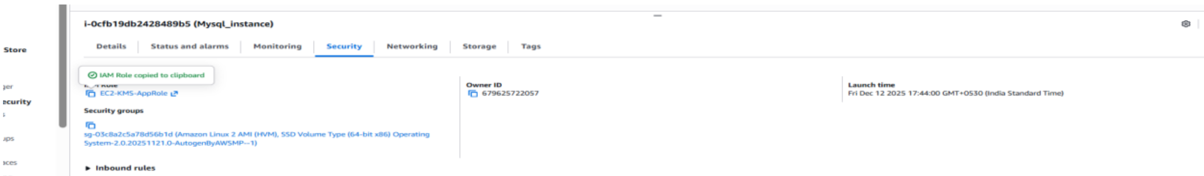


KMS TASK 3 — How do you restrict a KMS key so that only a specific IAM role can decrypt data?

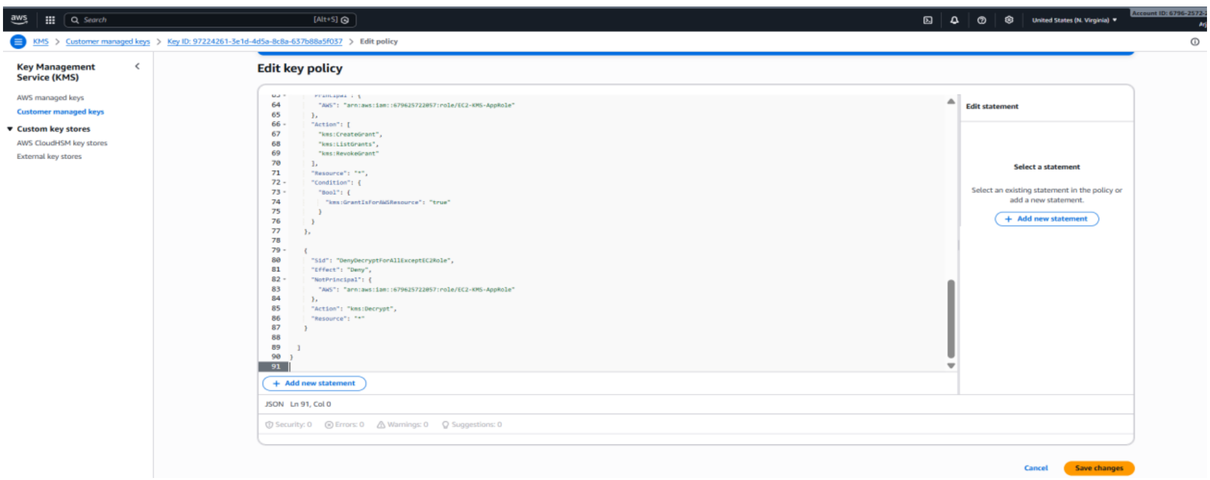
✅ **Steps to Follow (ONLY 7 Lines)**

1. Identify the application IAM role (EC2 or Lambda role).
2. Open AWS KMS → Customer-managed keys → select key.
3. Go to Key policy → Edit (JSON view).
4. Allow kms:Decrypt, Encrypt, GenerateDataKey to the app role.
5. Add an explicit Deny for all other principals.
6. Save the key policy and test access using the app role.
7. Confirm unauthorized users get AccessDenied.

Ec2 IAM role



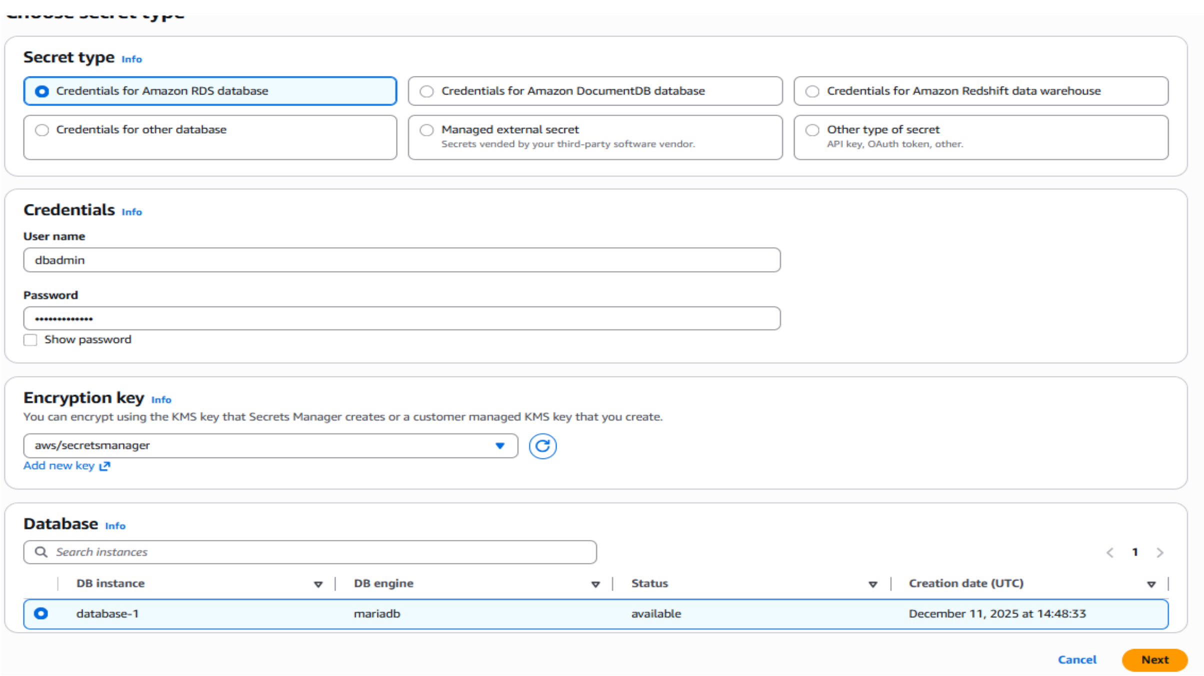
1. Open AWS KMS → Customer-managed keys → select key.
2. Go to Key policy → Edit (JSON view).
3. Allow kms:Decrypt, Encrypt, GenerateDataKey to the app role.
4. Add an explicit Deny for all other principals.
5. Save the key policy and test access using the app role.

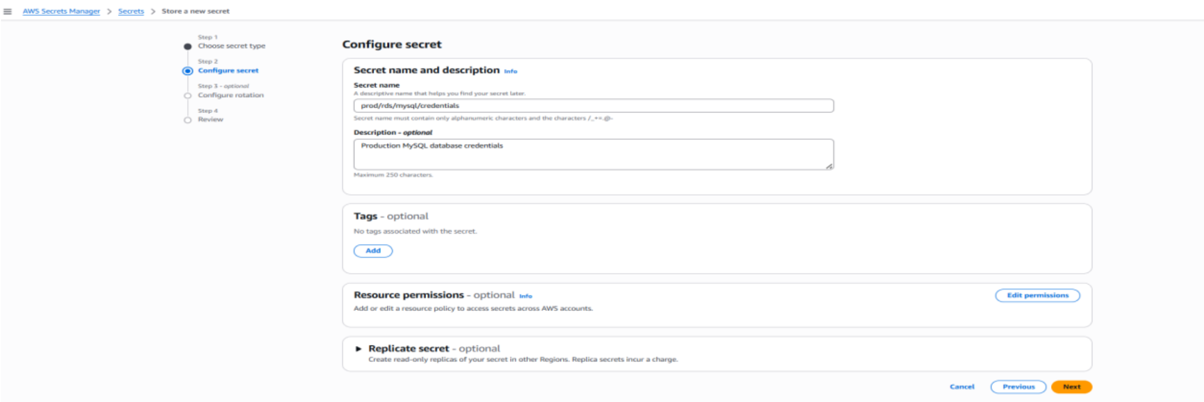


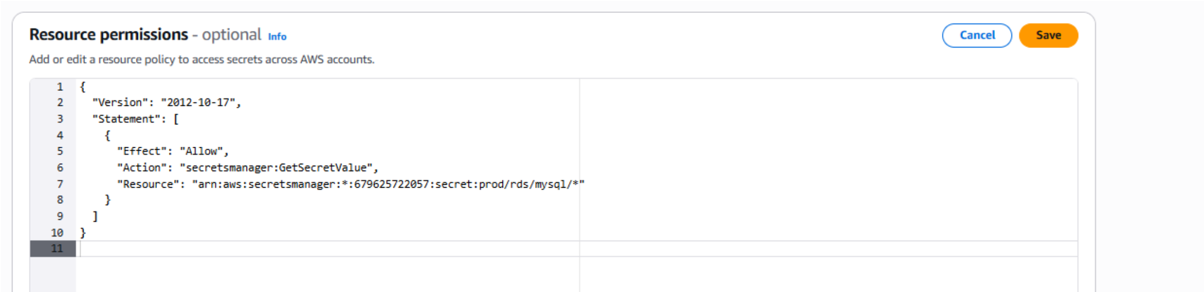
SECRETS MANAGER – How do you securely store database credentials using AWS Secrets Manager and allow an application to retrieve them?

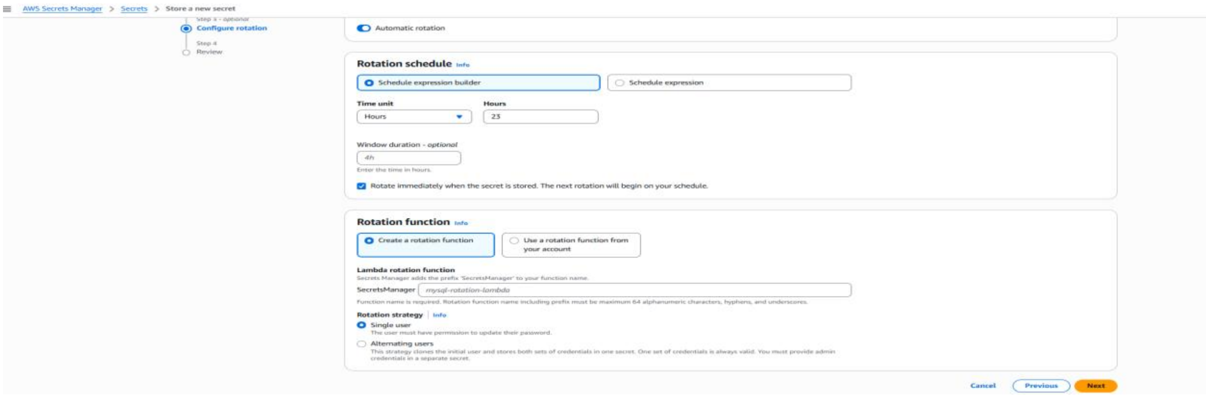
✅ Steps to Follow

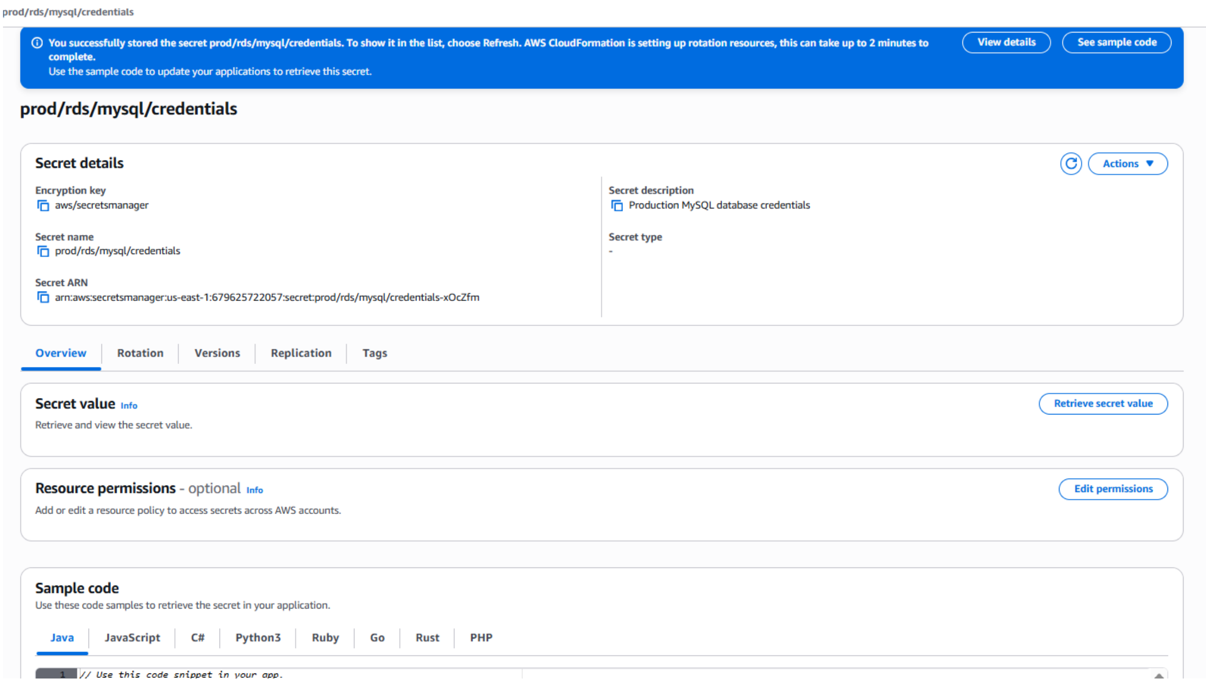
1. Create a customer-managed KMS key for Secrets Manager.
2. Create a secret storing DB username and password.
3. Encrypt the secret using the KMS CMK.
4. Name the secret using a standard path format.
5. Attach a Secrets Manager read policy to the app role.
6. Retrieve the secret from EC2 or Lambda.
7. Verify access using CloudTrail logs.




AWS Secrets Manager > Secrets > Store a new secret
Step 1
Choose secret type
Step 2
Configure secret
Step 3 - optional
Configure rotation
Step 4
Review
Configure secret
Secret name and description Info
Secret name
A descriptive name that helps you find your secret later.
prod/rds/mysql/credentials
Description - optional
Production MySQL database credentials
Maximum 250 characters.
Tags - optional
No tags associated with the secret.
Add
Resource permissions - optional Info
Add or edit a resource policy to access secrets across AWS accounts.
Edit permissions
Replicate secret - optional
Create read-only replicas of your secret in other Regions. Replica secrets incur a charge.
Cancel
Previous
Next


Resource permissions - optional Info
Cancel
Save
Add or edit a resource policy to access secrets across AWS accounts.
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Action": "secretsmanager:GetSecretValue",
"Resource": "arn:aws:secretsmanager:*:679625722057:secret:prod/rds/mysql/*"
}
]
}

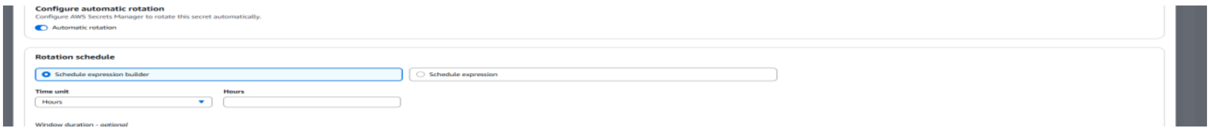

AWS Secrets Manager > Secrets > Store a new secret
Configure rotation
Step 4
Review
Automatic rotation
Rotation schedule Info
Schedule expression builder
Schedule expression
Time unit
Hours
Hours
23
Window duration - optional
4h
Enter the time in hours.
Rotate immediately when the secret is stored. The next rotation will begin on your schedule.
Rotation function Info
Create a rotation function
Use a rotation function from your account
Lambda rotation function
SecretsManager
mysql-rotation-lambda
Rotation strategy Info
Single user
The user must have permission to update their password.
Alternating users
Cancel
Previous
Next

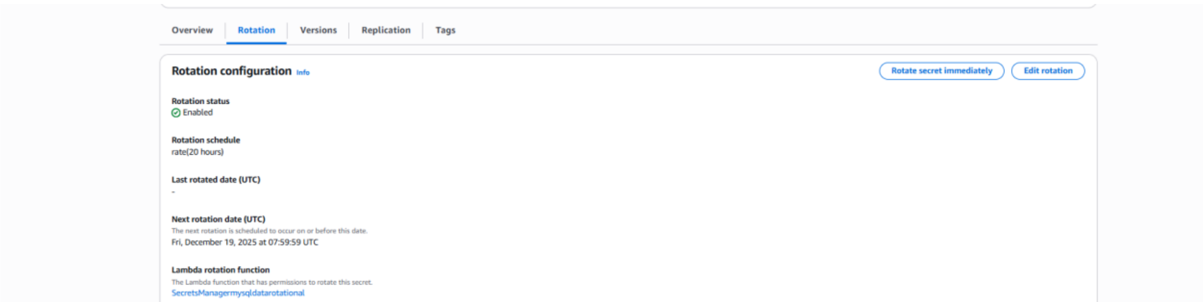

prod/rds/mysql/credentials
You successfully stored the secret prod/rds/mysql/credentials. To show it in the list, choose Refresh. AWS CloudFormation is setting up rotation resources, this can take up to 2 minutes to complete.
Use the sample code to update your applications to retrieve this secret.
View details
See sample code
prod/rds/mysql/credentials
Secret details
Actions
Encryption key
aws/secretsmanager
Secret name
prod/rds/mysql/credentials
Secret ARN
arn:aws:secretsmanager:us-east-1:679625722057:secret:prod/rds/mysql/credentials-xOcZfm
Secret description
Production MySQL database credentials
Secret type
-
Overview
Rotation
Versions
Replication
Tags
Secret value Info
Retrieve and view the secret value.
Retrieve secret value
Resource permissions - optional Info
Add or edit a resource policy to access secrets across AWS accounts.
Edit permissions
Sample code
Use these code samples to retrieve the secret in your application.
Java
JavaScript
C#
Python3
Ruby
Go
Rust
PHP
// Use this code snippet in your app.

SECRETS MANAGER TASK 2 —How do you enable automatic rotation of database credentials in AWS Secrets Manager?

✅ **Steps to Follow**

1. Open the existing database secret in Secrets Manager.
2. Enable automatic rotation.
3. Choose AWS-managed rotation Lambda for RDS.
4. Attach the Lambda to the secret.
5. Set rotation interval to 30 days.
6. Run Rotate secret now.





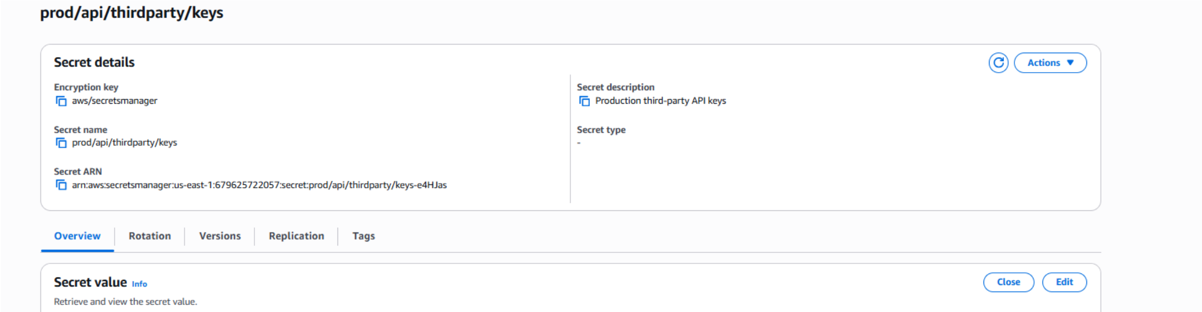
SECRETS MANAGER TASK 3-- How do you centrally manage third-party API keys using AWS Secrets Manager and allow applications to access them securely?"

✅ **Steps to Follow (Short – 7 Lines)**

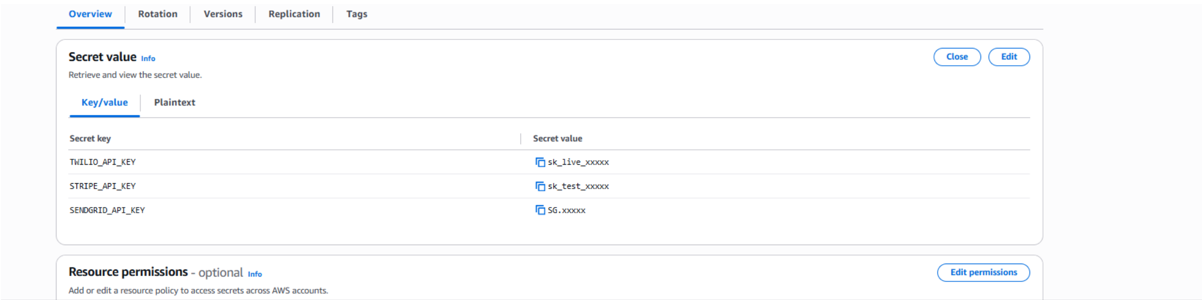
1. Create a secret to store third-party API keys.
2. Encrypt the secret using a KMS CMK.
3. Grant read-only access to application IAM roles.
4. Retrieve API keys dynamically from the app.
5. Rotate API keys by updating the secret.

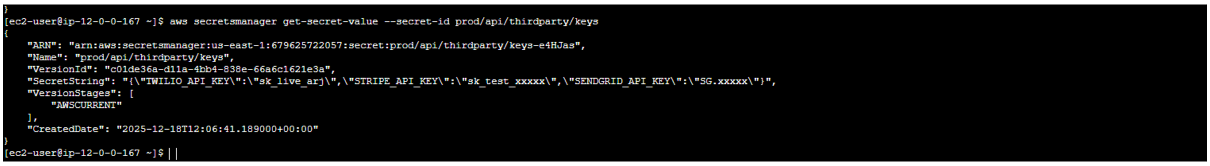
6. Verify apps use the new key without redeploy.

7. Audit secret access using CloudTrail.



```
       _/ _/
      _/m/'
[ec2-user@ip-12-0-0-167 ~]$ aws secretsmanager get-secret-value \
--secret-id prod/api/thirdparty/keys
3
Unable to locate credentials. You can configure credentials by running "aws login".
[ec2-user@ip-12-0-0-167 ~]$ aws configure
AWS Access Key ID [None]: AKIAZ4PGZRDE5II2TGOM
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]: json
[ec2-user@ip-12-0-0-167 ~]$ aws secretsmanager get-secret-value \
--secret-id prod/api/thirdparty/keys
{
    "ARN": "arn:aws:secretsmanager:us-east-1:679625722057:secret:prod/api/thirdparty/keys-e4HJas",
    "Name": "prod/api/thirdparty/keys",
    "VersionId": "c2b782f7-d3d5-4116-93ec-7ce4e85687b7",
    "SecretString": "{\"TWILIO_API_KEY\":\"sk_live_xxxxx\",\"STRIPE_API_KEY\":\"sk_test_xxxxx\",\"SENDGRID_API_KEY\":\"SG.xxxxx\"}",
    "VersionStages": [
        "AWSCURRENT"
    ],
    "CreatedDate": "2025-12-18T11:57:12.900000+00:00"
}
[ec2-user@ip-12-0-0-167 ~]$ ||
```





```
}
[ec2-user@ip-12-0-0-167 ~]$ aws secretsmanager get-secret-value --secret-id prod/api/thirdparty/keys
{
    "ARN": "arn:aws:secretsmanager:us-east-1:679625722057:secret:prod/api/thirdparty/keys-e4HJas",
    "Name": "prod/api/thirdparty/keys",
    "VersionId": "c01de36a-d11a-4bb4-838e-66a6c1621e3a",
    "SecretString": "{\"TWILIO_API_KEY\":\"sk_live_arj\",\"STRIPE_API_KEY\":\"sk_test_xxxxx\",\"SENDGRID_API_KEY\":\"SG.xxxxx\"}",
    "VersionStages": [
        "AWSCURRENT"
    ],
    "CreatedDate": "2025-12-18T12:06:41.189000+00:00"
}
[ec2-user@ip-12-0-0-167 ~]$ ||
```

Retrieved the old values

```
[ec2-user@ip-12-0-0-167 ~]$ aws secretsmanager get-secret-value --secret-id prod/api/thirdparty/keys
{
    "ARN": "arn:aws:secretsmanager:us-east-1:679625722057:secret:prod/api/thirdparty/keys-e4HJas",
    "Name": "prod/api/thirdparty/keys",
    "VersionId": "c01de36a-d11a-4bb4-838e-66a6c1621e3a",
    "SecretString": "{\"TWILIO_API_KEY\":\"sk_live_arj\",\"STRIPE_API_KEY\":\"sk_test_xxxxx\",\"SENDGRID_API_KEY\":\"SG.xxxxx\"}",
    "VersionStages": [
        "AWSCURRENT"
    ],
    "CreatedDate": "2025-12-18T12:06:41.189000+00:00"
}
[ec2-user@ip-12-0-0-167 ~]$ aws secretsmanager get-secret-value \
--secret-id prod/api/thirdparty/keys \
--version-stage AWSPREVIOUS
{
    "ARN": "arn:aws:secretsmanager:us-east-1:679625722057:secret:prod/api/thirdparty/keys-e4HJas",
    "Name": "prod/api/thirdparty/keys",
    "VersionId": "c2b782f7-d3d5-4116-93ec-7ce4e85687b7",
    "SecretString": "{\"TWILIO_API_KEY\":\"sk_live_xxxxx\",\"STRIPE_API_KEY\":\"sk_test_xxxxx\",\"SENDGRID_API_KEY\":\"SG.xxxxx\"}",
    "VersionStages": [
        "AWSPREVIOUS"
    ],
    "CreatedDate": "2025-12-18T11:57:12.900000+00:00"
}
[ec2-user@ip-12-0-0-167 ~]$ ||
```